

WORKFLOW_PARTE_1

September 25, 2025

<CENTER>

</CENTER>

<CENTER>DATA SCIENCE PROJECT</CENTER>

<CENTER>WORKFLOW PARTE 1 DO PROJETO</CENTER>

NOME COMPLETO: Luiz Felipe Pereira Figueiredo

LINK PARA SEU PRÓPRIO COLAB: [Aqui](#)

LINK PARA SEU PRÓPRIO GITHUB: [Aqui](#)

LINK PARA SEU VÍDEO NO YOUTUBE: [Aqui](#)

1 Cenário Fictício:

Prezados(as) alunos(as),

É com entusiasmo que apresento a proposta do **Projeto de Conclusão de Curso (TCC)** para os estudantes do curso de Ciência de Dados da Universidade Vila Velha: o UWine – Clube de Vinhos da UVV.

A proposta do projeto consiste na análise de um Clube de Vinhos universitário fictício, com base em dados simulados, que será totalmente orientado por técnicas e práticas de Ciência de Dados.

Sendo assim, você foi designado como **Consultor em Ciência de Dados** (*Data Science Consultant*) para fazer uma análise completa dos dados dos clientes a partir de um conjunto de dados (Amostra) com mais de 1 milhão de notas fiscais (~ 350 Megabytes). Além disso, há uma entrada diária de aproximadamente umas 5000 novas notas fiscais no sistema.

O objetivo é permitir que cada consultor(a) percorra foco analíticos diferentes dentro do mesmo ecossistema de dados, aplicando modelos de machine learning supervisionado ou não supervisionado, análise preditiva, dashboards interativos, entre outros.

Para garantir organização, profundidade técnica e coerência metodológica no desenvolvimento do **Projeto UWine**, adotaremos um workflow composto por cinco etapas principais, alinhadas às boas práticas da Ciência de Dados. Cada etapa representa um marco do processo analítico, permitindo

que cada analista avance de forma estruturada do entendimento inicial dos dados até a entrega final dos insights.

2 BIBLIOTECAS E FUNÇÕES E CÓDIGOS

```
[15]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from typing import Tuple
from scipy import stats

from scipy.stats import (
    anderson, shapiro, normaltest,
    pearsonr, spearmanr, kendalltau, chi2_contingency,
    ttest_ind, ttest_rel, f_oneway,
    mannwhitneyu, wilcoxon, kruskal, ks_2samp, levene, bootstrap
)
pd.set_option('display.max_columns', None)
sns.set_theme(style="whitegrid")
```

```
[9]: # load do seu TABLE ? aqui: Numero ???
df = pd.read_csv('table6.csv')
```

```
[10]: # CORREÇÃO DO ERRO ETL:
def troca_rotulos(df: pd.DataFrame) -> pd.DataFrame:
    colunas = list(df.columns)
    colunas[3], colunas[4] = colunas[4], colunas[3]
    df.columns = colunas
    return df

df = troca_rotulos(df)

df = df.drop(columns=['index', 'Unnamed: 0'], axis=0)
```

```
[4]: df.head()
```

```
[4]:  NOTA FISCAL  SEXO  REGIÃO  ESTADO CIVIL  TIPO DA CONTA  RENDA BRUTO (R$) \
0      8233915     1       2             1             1           1663.00
1      8936040     2       2             1             2           1777.14
2      5808769     1       2             1             1           1677.57
3      8518924     3       2             3             1          18548.60
4      4282314     1       2             1             1           1714.19

                                OPINIÃO DO CLIENTE  alcohol  malic_acid \
0      Gostei do atendimento prestado pela UVVine  12.490431    2.308571
1  Funcionou bem e me atendeu no prazo certo a UV...  12.971861    2.857346
```

2	Fiquei satisfeito com o serviço da UVVine	13.775579	1.643997
3	Nada demais, serviço regular na UVVine	11.348116	2.278516
4	Gostei do atendimento prestado pela UVVine	11.367704	-0.142202

	ash	alkalinity_of_ash	magnesium	total_phenols	flavanoids	\
0	2.301077	26.140638	121.037802	1.533384	3.751384	
1	2.427161	23.622779	99.556351	3.212802	4.060955	
2	2.219302	19.065256	88.947058	2.917240	0.981155	
3	2.479980	15.438017	84.594798	2.764763	2.128939	
4	2.699032	20.313369	103.619877	1.837121	0.955861	

	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	\
0	0.488961	2.421304	3.807632	0.824571	
1	0.362116	1.297826	4.772804	1.346393	
2	0.332140	1.589915	3.422146	0.912559	
3	0.413808	0.894238	5.892124	1.134504	
4	0.325568	0.895206	6.425832	1.079519	

	od280/od315_of_diluted_wines	proline	TOTAL (R\$)
0	3.344343	1126.735368	1531.95
1	3.167014	893.502173	1708.04
2	3.187565	655.179237	1545.35
3	2.733674	420.205151	17808.65
4	2.931656	858.538658	1750.46

```
[11]: df.shape
```

```
[11]: (1120000, 21)
```

```
[13]: def statistic(_dataset: pd.Series) -> pd.Series:
    """
    Retorna medidas de resumo da série: describe (sem count), range e moda,
    com arredondamento final.
    """
    _describe = _dataset.describe()
    _describe = _describe.drop('count')
    _describe.loc['range'] = _describe['max'] - _describe['min']
    _describe.loc['mode'] = stats.mode(_dataset, keepdims=False)[0]
    _describe = np.around(_describe.sort_values(), 2)
    return _describe

def PlotarStatistic(dataset: pd.Series, ylabel: str = None, title_suffix: str =
↳ ""):
    """
    Plota medidas de resumo (média/mediana/moda) e imprime a tabela descritiva.
    """
```

```

_describe = statistic(dataset)

mean_mu    = 'MÉDIA:  $\overline{\mu}$  = ' + f'[_describe["mean"]:.2f]'
median_mu  = 'MEDIANA:  $\tilde{\mu}$  = ' + f'[_describe["50%"]:.2f]'
mode_mu    = 'MODA:  $\hat{\mu}$  = ' + f'[_describe["mode"]:.2f]'

# linha principal em azul
dataset.agg(statistic).plot(figsize=[15, 6], color="royalblue",
↪label='Medidas de resumo')

# pontos destacados com cores distintas
plt.plot(list(_describe.index).index('mean'), _describe["mean"], 'D',
↪color="crimson", label=mean_mu)
plt.plot(list(_describe.index).index('50%'), _describe["50%"], 'D',
↪color="forestgreen", label=median_mu)
plt.plot(list(_describe.index).index('mode'), _describe["mode"], 'D',
↪color="darkorange", label=mode_mu)

plt.title(f'Estatística Descritiva do Dataset {title_suffix}')
plt.legend()
plt.ylabel(yLabel)
plt.show()

print('Estatística descritiva do dataset:')
print(_describe)

def Bootstrap_Distribution(data: np.ndarray, metric=np.mean, bins: int = 25, ci:
↪ float = 0.95, seed: int = 42):
    """
    Exibe histograma da distribuição bootstrap da estatística (média por
↪ padrão),
    com linhas verticais nos limites do IC e imprime IC e erro-padrão.
    """
    rng = np.random.default_rng(seed)
    data = (np.asarray(data),)
    res = bootstrap(data, metric, confidence_level=ci, random_state=rng)

    fig, ax = plt.subplots(figsize=(8,5))
    ax.hist(res.bootstrap_distribution, bins, color="royalblue",
↪edgecolor="black", alpha=0.7)
    ax.set_title('Distribuição Bootstrap')
    ax.set_xlabel('Valor da Estatística')
    ax.set_ylabel('Frequência')

    # linhas do intervalo de confiança

```

```

ax.vlines(res.confidence_interval.low, 0, ax.get_ylim()[1],
          linestyle='--', colors="crimson", label="Limite inferior IC")
ax.vlines(res.confidence_interval.high, 0, ax.get_ylim()[1],
          linestyle='--', colors="forestgreen", label="Limite superior IC")

ax.legend()
plt.tight_layout()
plt.show()

print('Intervalo de Confiança (95%):')
print(f'LOW : {res.confidence_interval.low: .2f}')
print(f'HIGH: {res.confidence_interval.high: .2f}')
print(f'SE   : {res.standard_error: .2f}')
return res

def SE(dataset: pd.Series) -> float:
    """
    Erro-Padrão (SE) da média de uma série numérica.
    """
    return float(dataset.std(ddof=1)) / (len(dataset) ** 0.5)

def find_stabilization_n(ns, se_values, rel_drop_threshold=0.01, window=5):
    """
    Heurística de estabilização: define n* quando a redução relativa média do SE
    (média móvel) cai abaixo de um limiar. Ex.: 1% por passo => curva
    ↪ "achatando".
    """
    se = pd.Series(se_values, index=ns, dtype=float)
    pct_drop = -(se.pct_change().fillna(0.0))
    smooth = pct_drop.rolling(window=window, min_periods=1).mean()
    cand = smooth[smooth < rel_drop_threshold]
    if len(cand) == 0:
        return None
    return int(cand.index[0])

def bootstrap_ci_halfwidth(sample_values: np.ndarray, n: int, B: int = 1000,
    ↪ alpha: float = 0.05,
                                stat=np.mean, seed=42):
    """
    Meia-largura do IC (percentil) via bootstrap para uma subamostra de tamanho
    ↪ n.
    """
    rng = np.random.default_rng(seed)

```

```

vals = np.asarray(sample_values)
stats_boot = []
for _ in range(B):
    s = rng.choice(vals, size=n, replace=True)
    stats_boot.append(stat(s))
lo = np.percentile(stats_boot, 100 * (alpha/2))
hi = np.percentile(stats_boot, 100 * (1 - alpha/2))
return (hi - lo) / 2.0

def pick_n_for_relative_margin(values: np.ndarray, rel_margin=0.05, alpha=0.05,
    grid=None,
                                stat=np.mean, seed=42):
    """
    Dado um alvo de precisão relativa (ex.: 5% da média), varre um grid de n e
    retorna tabela com half-width vs. target, além do menor n que atende a meta.
    """
    if grid is None:
        grid = [1_000, 2_000, 4_000, 8_000, 12_000, 20_000, 40_000, 80_000]
    mu = float(stat(values))
    target = rel_margin * mu
    rows = []
    for n in grid:
        hw = bootstrap_ci_halfwidth(values, n=n, B=600, alpha=alpha, stat=stat,
            seed=seed)
        rows.append((n, hw, target, hw <= target))
    out = pd.DataFrame(rows, columns=["n", "half_width", "target_half_width",
        "meets"])
    ok = out[out["meets"]].sort_values("n")
    n_min = int(ok.iloc[0]["n"]) if not ok.empty else None
    return out, n_min

```

3 Workflow: Detalhamento das Atividades

3.1 Atividade A1 ([Link](#)): Estratificação amostral da população

3.1.1 Atividade A1.1: Determinar o tamanho da amostra

```

[16]: def AtividadeA1_1(x_series: pd.Series,
    target_label: str = "TOTAL (R$)",
    n_inicial: int = 1_000,
    n_final: int = 100_000,
    passo: int = 1_000,
    rel_drop_threshold: float = 0.01,
    window: int = 5,
    n_piloto: int | None = None,
    show_zoom_share: float = 0.10):

```

```

"""
Atividade A1.1 - Tamanho da Amostra

Essa função calcula o tamanho de amostra necessário para estimar a média
de uma variável numérica de forma estável. A ideia é:
- rodar estatística descritiva,
- aplicar bootstrap para visualizar intervalo de confiança,
- varrer tamanhos de amostra e calcular o erro-padrão (SE),
- identificar onde o SE estabiliza,
- verificar um critério de precisão relativa ( $\pm 5\%$  da média),
- gerar gráficos gerais e de zoom.

Parâmetros
-----
x_series : pd.Series
    Série numérica da variável alvo já filtrada/limpa.
target_label : str
    Nome da variável alvo para exibição nos gráficos.
n_inicial : int
    Valor inicial da varredura de n.
n_final : int
    Valor final da varredura de n.
passo : int
    Incremento da varredura.
rel_drop_threshold : float
    Limiar de queda relativa do SE para considerar estabilização.
window : int
    Tamanho da janela da média móvel no cálculo de estabilização.
n_piloto : int | None
    Amostra piloto usada no bootstrap inicial.
show_zoom_share : float
    Percentual final da curva usado no gráfico de zoom.

Retorna
-----
dict
    Contém n_star, tabela de precisão relativa, n_min_5pct,
    vetores ns e array_se, resultado do bootstrap piloto e N_POP.
"""
# preparo da série (garantindo numérica e sem negativos)
x_full = pd.to_numeric(pd.Series(x_series), errors="coerce").dropna().
↳ astype(float)
x_full = x_full[x_full >= 0]
N_POP = len(x_full)
print(f"Tamanho disponível (pós-limpeza) para '{target_label}': {N_POP:,}↳
↳ observações.")

```

```

# estatística descritiva
PlotarStatistic(x_full, yLabel=target_label, title_suffix=f"-_{target_label}")

# bootstrap inicial em amostra piloto
if n_piloto is None:
    n_piloto = 20_000 if N_POP >= 20_000 else max(1000, int(N_POP * 0.8))
res_boot = Bootstrap_Distribution(
    x_full.sample(n=n_piloto, replace=False, random_state=42).values,
    metric=np.mean, bins=30, ci=0.95, seed=42
)

# curva SE vs n
ns = list(range(n_inicial, min(n_final, N_POP) + 1, passo))
array_se = np.zeros(len(ns), dtype=float)
for j, n in enumerate(ns):
    tb = x_full.sample(n=n, replace=False, random_state=42 + j)
    array_se[j] = SE(tb)

plt.figure(figsize=(10,5))
plt.plot(ns, array_se, marker='o', color='royalblue', label="SE vs n")
if len(array_se) >= 10:
    plt.hlines(np.mean(array_se[-10:]), ns[0], ns[-1],
               linestyle='--', colors='darkorange',
               label="SE médio últimos pontos")
plt.title(f"Erro-Padrão (SE) da média de {target_label} vs. n")
plt.ylabel('Erro-Padrão')
plt.xlabel('Tamanho da amostra (n)')
plt.legend()
plt.tight_layout()
plt.show()

if len(array_se) >= 10:
    print(f"SE médio nas últimas janelas (estabilização aproximada): {np.
    mean(array_se[-10:]).4f}")

# sugestão de n* pelo achatamento do SE
n_star = find_stabilization_n(ns, array_se,
    rel_drop_threshold=rel_drop_threshold, window=window)
print(f"Sugestão automática de n* (achatamento do SE_{int(rel_drop_threshold*100)}%): {n_star}")

# critério de ±5% da média
grid = [1_000, 2_000, 4_000, 8_000, 12_000, 20_000, 40_000, 80_000]
table_ci, n_min_5 = pick_n_for_relative_margin(x_full.values, rel_margin=0.05, alpha=0.05, grid=grid)
print(table_ci)

```

```

print(f"n mínimo para  $\pm 5\%$  da média (IC 95%): {n_min_5}")

# gráfico de zoom nos últimos pontos
zoom_len = max(1, int(len(ns) * show_zoom_share))
ns_zoom = ns[-zoom_len:]
se_zoom = array_se[-zoom_len:]

plt.figure(figsize=(10,5))
plt.plot(ns_zoom, se_zoom, marker='s', color='crimson', label="SE vs n_
↪(zoom)")
plt.hlines(np.mean(se_zoom), ns_zoom[0], ns_zoom[-1],
           linestyle='--', colors='forestgreen',
           label="SE médio zoom")
plt.title(f"Erro-Padrão (SE) da média de {target_label} - Zoom_
↪({int(show_zoom_share*100)}% finais)")
plt.ylabel('Erro-Padrão')
plt.xlabel('Tamanho da amostra (n)')
plt.grid(True, linestyle="--", alpha=0.7)
plt.legend()
plt.tight_layout()
plt.show()

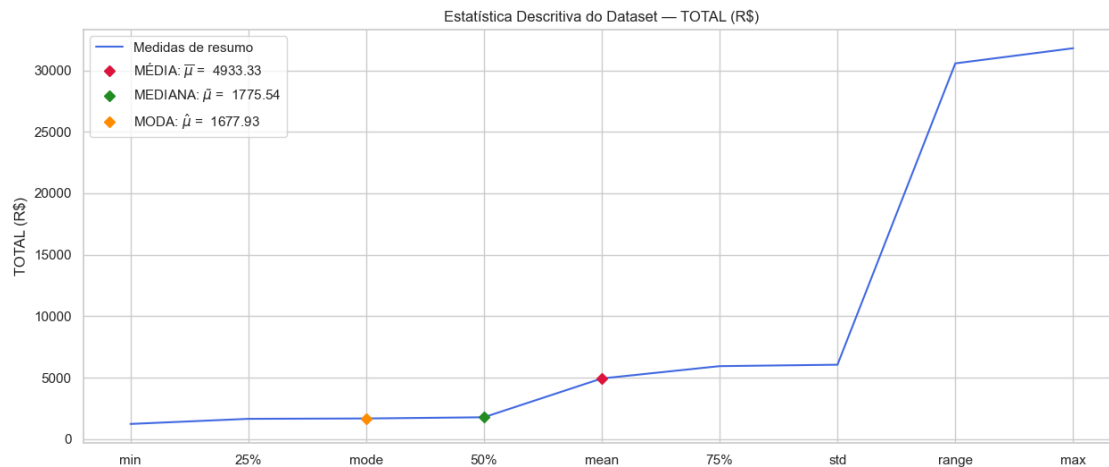
print(f"SE médio na região de zoom (últimos {len(se_zoom)} pontos): {np.
↪mean(se_zoom):.4f}")

return {
    "n_star": n_star,
    "table_ci_5pct": table_ci,
    "n_min_5pct": n_min_5,
    "ns": ns,
    "array_se": array_se,
    "pilot_bootstrap": res_boot,
    "N_POP": N_POP
}

resultados = AtividadeA1_1(
    x_series=df["TOTAL (R$)"],
    target_label="TOTAL (R$)",
    n_inicial=1_000, n_final=100_000, passo=1_000,
    rel_drop_threshold=0.01, window=5,
    n_piloto=None, show_zoom_share=0.10
)

```

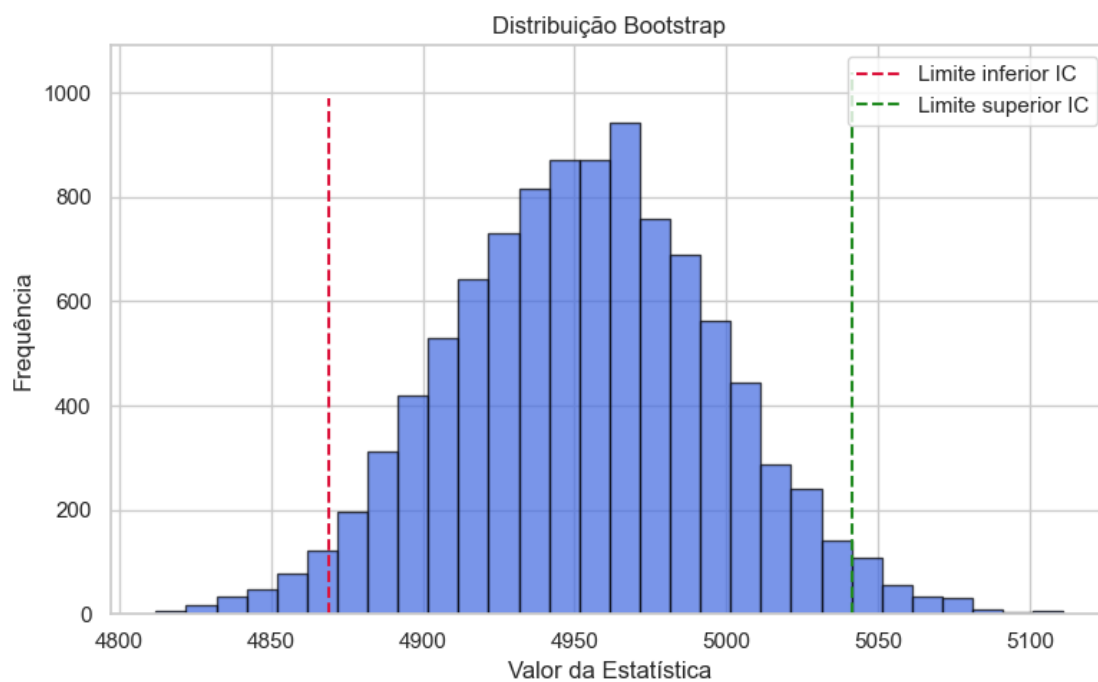
Tamanho disponível (pós-limpeza) para 'TOTAL (R\$)': 1,120,000 observações.



Estadística descritiva do dataset:

```
min      1231.71
25%      1648.20
mode     1677.93
50%      1775.54
mean     4933.33
75%      5935.58
std      6050.04
range    30570.88
max      31802.59
```

Name: TOTAL (R\$), dtype: float64

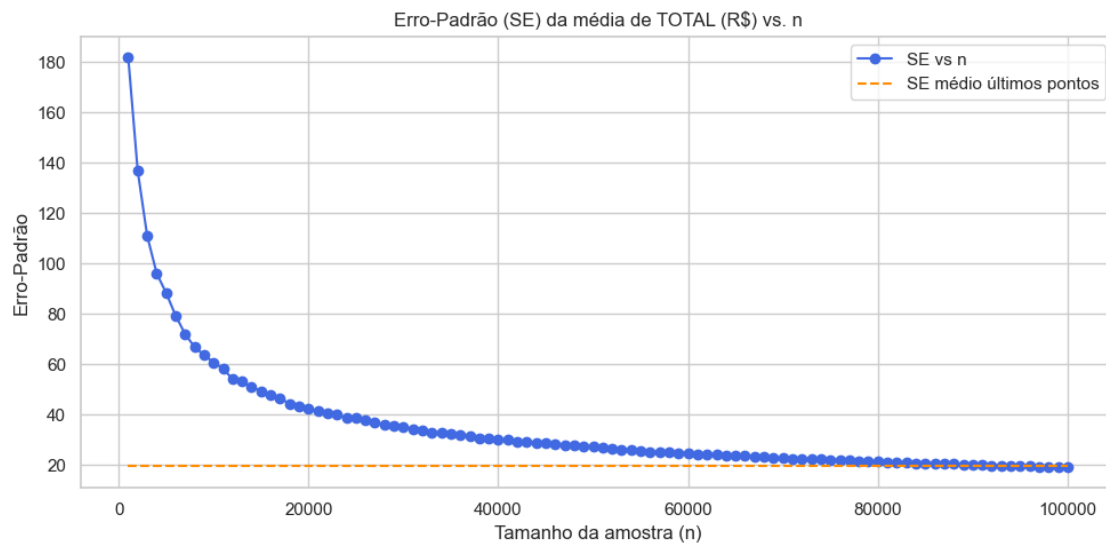


Intervalo de Confiança (95%):

LOW : 4869.02

HIGH: 5041.54

SE : 44.02

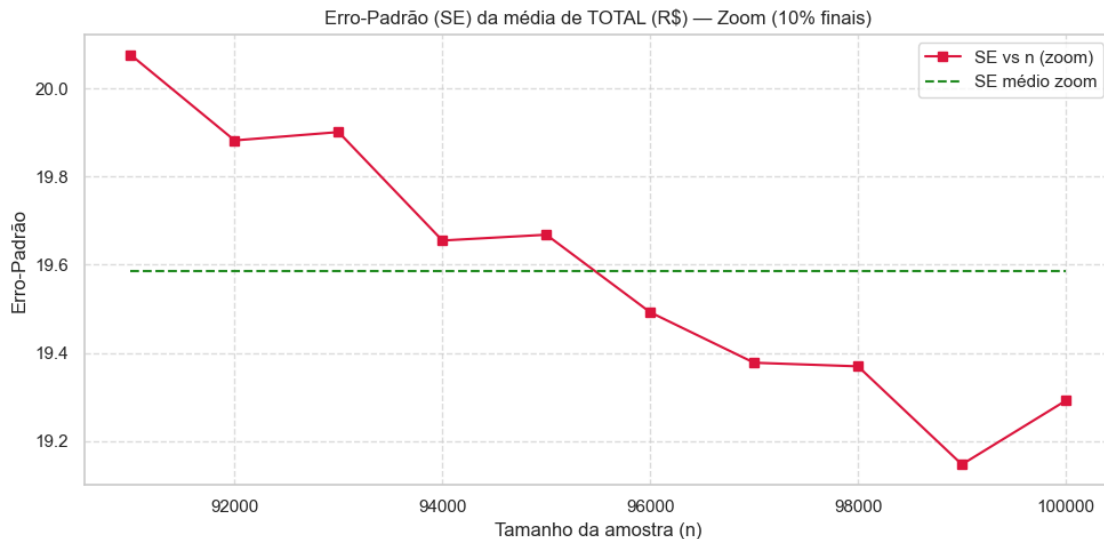


SE médio nas últimas janelas (estabilização aproximada): 19.5861

Sugestão automática de n* (achatamento do SE ~1%): 1000

	n	half_width	target_half_width	meets
0	1000	376.934264	246.66651	False
1	2000	266.005949	246.66651	False
2	4000	185.024877	246.66651	True
3	8000	127.710760	246.66651	True
4	12000	107.243477	246.66651	True
5	20000	85.378668	246.66651	True
6	40000	61.739172	246.66651	True
7	80000	42.801879	246.66651	True

n mínimo para $\pm 5\%$ da média (IC 95%): 4000



SE médio na região de zoom (últimos 10 pontos): 19.5861

A partir da análise dos gráficos, considerando o método de estabilização da curva e o bootstrap para cálculo do erro-padrão, observa-se que o tamanho adequado da amostra se encontra entre 95 mil e 96 mil observações. Para fins de continuidade, adota-se 95,5 mil como valor de referência.

3.2 #Atividade A1.2: Analisar os outliers

```
[17]: def AtividadeA1_2(x_series: pd.Series, var_label: str = ""):
    """
    Atividade A1.2 - Analisar os Outliers

    Recebe uma série numérica e identifica outliers usando o método do IQR.
    Exibe boxplot, histograma e estatísticas resumidas.

    Parâmetros
    -----
    x_series : pd.Series
        Série numérica da variável a ser analisada.
    var_label : str
        Nome da variável (usado nos títulos dos gráficos).

    Retorna
    -----
    dict
        Contém limites inferiores e superiores, contagem de outliers
        inferiores/superiores e proporção em relação ao total.
    """
    x = pd.to_numeric(x_series, errors="coerce").dropna().astype(float)
```

```

Q1 = x.quantile(0.25)
Q3 = x.quantile(0.75)
IQR = Q3 - Q1
lim_inf = Q1 - 1.5 * IQR
lim_sup = Q3 + 1.5 * IQR

outliers_inf = (x < lim_inf).sum()
outliers_sup = (x > lim_sup).sum()
total_outliers = outliers_inf + outliers_sup
prop_outliers = total_outliers / len(x)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
sns.boxplot(x=x, ax=axes[0], color="royalblue")
axes[0].set_title(f"Boxplot - {var_label}")

sns.histplot(x, kde=True, ax=axes[1], color="crimson")
axes[1].axvline(lim_inf, color="black", linestyle="--", label="Limite_
↳inferior")
axes[1].axvline(lim_sup, color="black", linestyle="--", label="Limite_
↳superior")
axes[1].legend()
axes[1].set_title(f"Histograma - {var_label}")
plt.tight_layout()
plt.show()

print(f"Variável: {var_label}")
print(f"Q1 = {Q1:.2f} | Q3 = {Q3:.2f} | IQR = {IQR:.2f}")
print(f"Limite inferior = {lim_inf:.2f} | Limite superior = {lim_sup:.2f}")
print(f"Outliers abaixo: {outliers_inf}")
print(f"Outliers acima : {outliers_sup}")
print(f"Total de outliers: {total_outliers} ({prop_outliers:.2%} do total)")

return {
    "Q1": Q1,
    "Q3": Q3,
    "IQR": IQR,
    "lim_inf": lim_inf,
    "lim_sup": lim_sup,
    "outliers_inf": outliers_inf,
    "outliers_sup": outliers_sup,
    "total_outliers": total_outliers,
    "prop_outliers": prop_outliers
}

```

```

[18]: # lista de variáveis obrigatórias e relevantes
vars_outliers = [

```

```

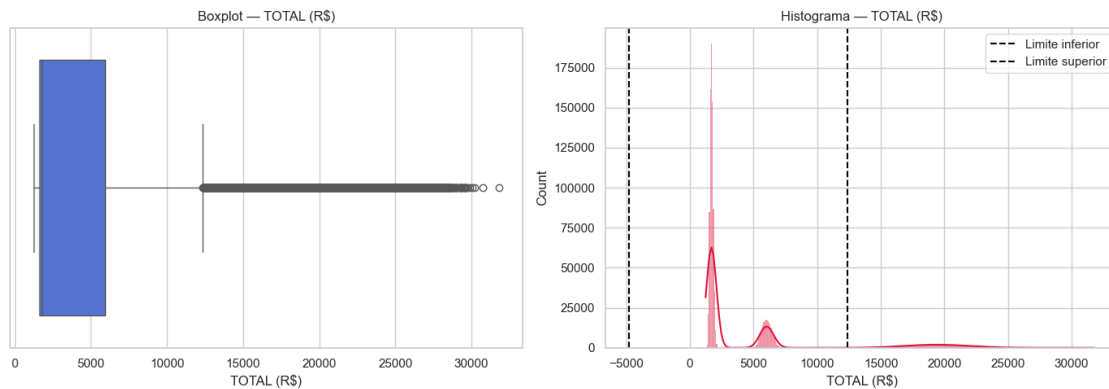
    "TOTAL (R$)",
    "RENDA BRUTO (R$)",
    "alcohol",
    "flavanoids",
    "color_intensity",
    "proline"
]

# dicionário para salvar os resultados
resultados_outliers = {}

# execução para cada variável
for col in vars_outliers:
    print("="*60)
    print(f"Analisando outliers em: {col}")
    resultados_outliers[col] = AtividadeA1_2(df[col], var_label=col)

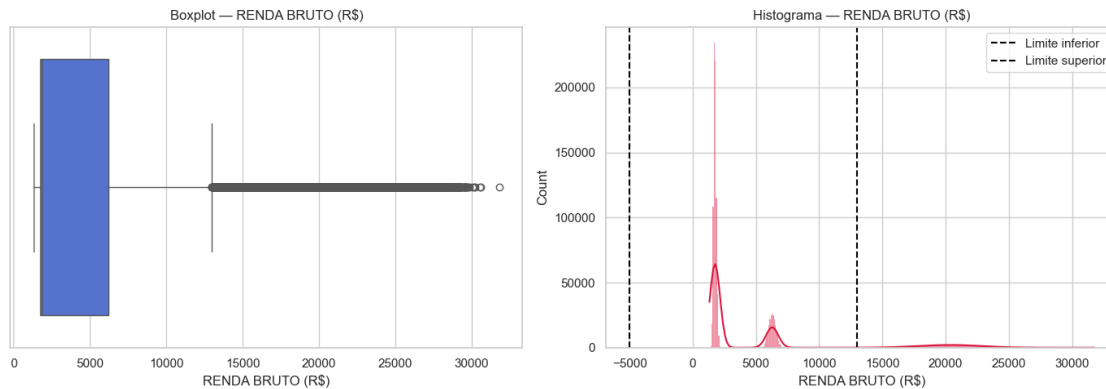
```

=====
Analisando outliers em: TOTAL (R\$)



Variável: TOTAL (R\$)
Q1 = 1648.20 | Q3 = 5935.58 | IQR = 4287.38
Limite inferior = -4782.87 | Limite superior = 12366.65
Outliers abaixo: 0
Outliers acima : 149745
Total de outliers: 149745 (13.37% do total)

=====
Analisando outliers em: RENDA BRUTO (R\$)



Variável: RENDA BRUTO (R\$)

Q1 = 1724.84 | Q3 = 6220.81 | IQR = 4495.97

Limite inferior = -5019.11 | Limite superior = 12964.76

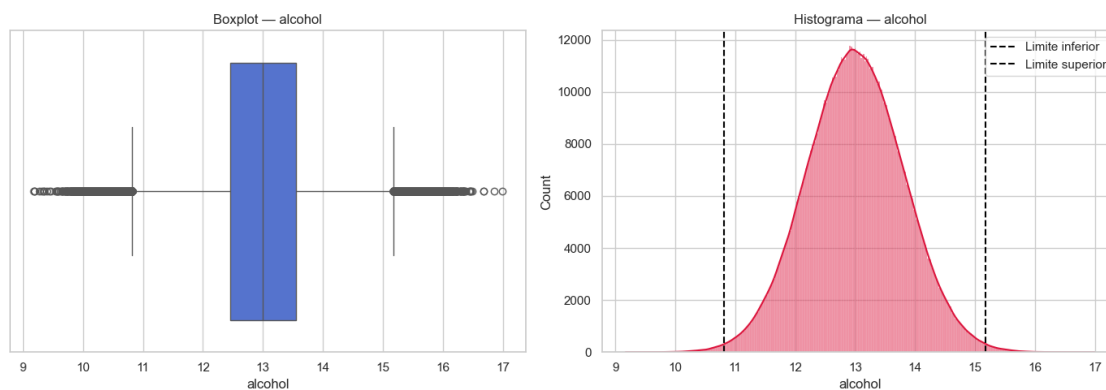
Outliers abaixo: 0

Outliers acima : 149795

Total de outliers: 149795 (13.37% do total)

=====

Analisando outliers em: alcohol



Variável: alcohol

Q1 = 12.46 | Q3 = 13.55 | IQR = 1.09

Limite inferior = 10.82 | Limite superior = 15.18

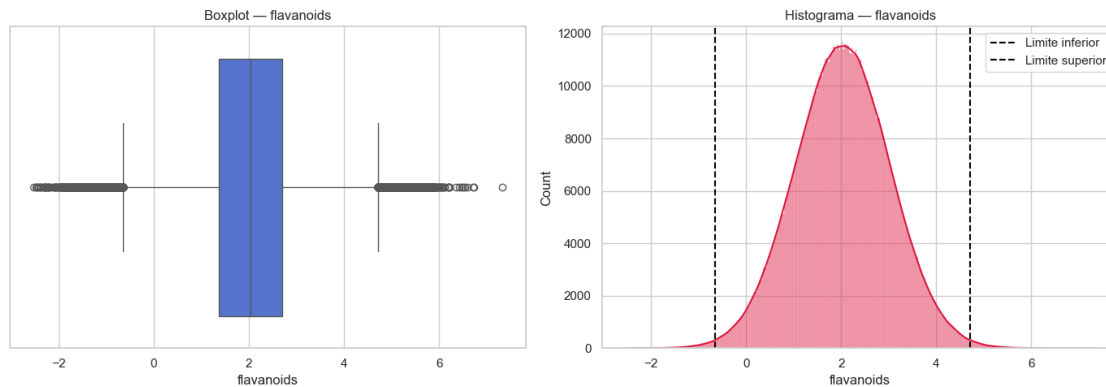
Outliers abaixo: 4022

Outliers acima : 3922

Total de outliers: 7944 (0.71% do total)

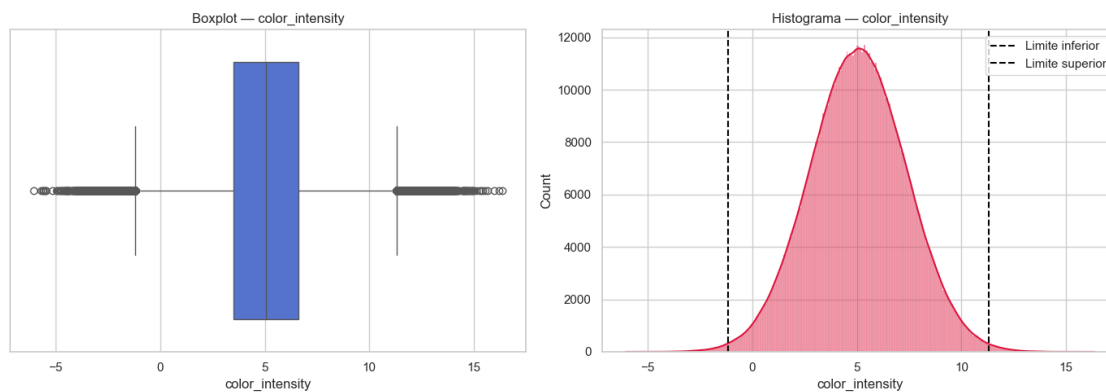
=====

Analisando outliers em: flavanoids



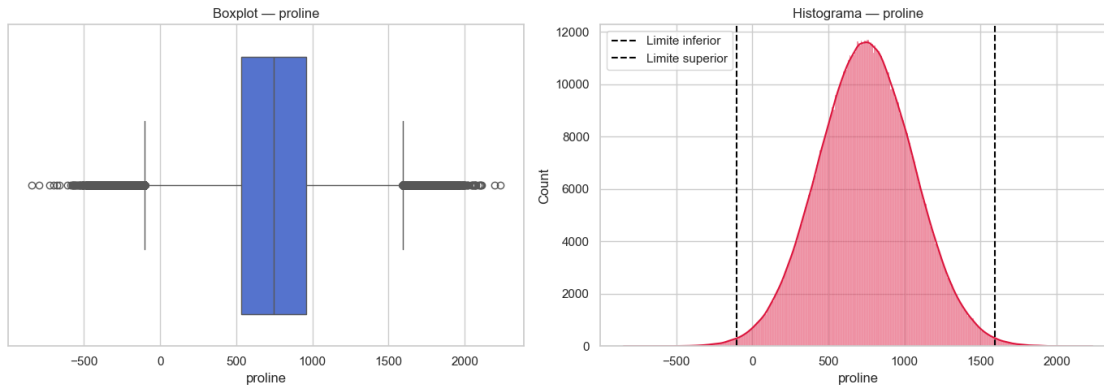
Variável: flavanoids
 Q1 = 1.36 | Q3 = 2.70 | IQR = 1.34
 Limite inferior = -0.65 | Limite superior = 4.71
 Outliers abaixo: 3907
 Outliers acima : 3918
 Total de outliers: 7825 (0.70% do total)

=====
 Analisando outliers em: color_intensity



Variável: color_intensity
 Q1 = 3.50 | Q3 = 6.62 | IQR = 3.12
 Limite inferior = -1.17 | Limite superior = 11.29
 Outliers abaixo: 3920
 Outliers acima : 3912
 Total de outliers: 7832 (0.70% do total)

=====
 Analisando outliers em: proline



Variável: proline

Q1 = 535.23 | Q3 = 958.91 | IQR = 423.69

Limite inferior = -100.30 | Limite superior = 1594.44

Outliers abaixo: 3997

Outliers acima : 3863

Total de outliers: 7860 (0.70% do total)

```
[19]: def flag_outliers_iqr(x_series: pd.Series, k: float = 1.5) -> pd.Series:
    """
    Cria flag de outliers (0 = normal, 1 = outlier) pelo método do IQR.

    Parâmetros
    -----
    x_series : pd.Series
        Série numérica da variável.
    k : float
        Multiplicador do IQR (1.5 = padrão do boxplot).

    Retorna
    -----
    pd.Series
        Série de flags com 0 (normal) e 1 (outlier).
    """
    x = pd.to_numeric(x_series, errors="coerce").dropna().astype(float)
    Q1 = x.quantile(0.25)
    Q3 = x.quantile(0.75)
    IQR = Q3 - Q1
    lim_inf = Q1 - k * IQR
    lim_sup = Q3 + k * IQR
    return (x_series < lim_inf) | (x_series > lim_sup)

# aplica no dataframe
df["OUTLIER_TOTAL"] = flag_outliers_iqr(df["TOTAL (R$)"]).astype(int)
```

```
df["OUTLIER_RENDA"] = flag_outliers_iqr(df["RENDA BRUTO (R$)"]).astype(int)

# checar resultado
print(df[["TOTAL (R$)", "OUTLIER_TOTAL", "RENDA BRUTO (R$)", "OUTLIER_RENDA"]].
      head())
print("Proporção de outliers em TOTAL:", df["OUTLIER_TOTAL"].mean())
print("Proporção de outliers em RENDA:", df["OUTLIER_RENDA"].mean())
```

	TOTAL (R\$)	OUTLIER_TOTAL	RENDA BRUTO (R\$)	OUTLIER_RENDA
0	1531.95	0	1663.00	0
1	1708.04	0	1777.14	0
2	1545.35	0	1677.57	0
3	17808.65	1	18548.60	1
4	1750.46	0	1714.19	0

Proporção de outliers em TOTAL: 0.13370089285714284

Proporção de outliers em RENDA: 0.1337455357142857

A análise de outliers evidenciou que aproximadamente 13% dos clientes apresentam valores extremos em TOTAL (R)*e*RENDA BRUTO (R). Esses casos refletem consumidores de alto ticket ou renda elevada, que podem constituir um segmento premium do clube, sendo relevante preservá-los na base. Já nas variáveis físico-químicas do vinho, foram identificados poucos outliers, o que sugere uma distribuição mais homogênea. Para a Parte 1, optou-se por apenas reportar os outliers, mantendo-os na base. Na Parte 2, quando forem aplicadas técnicas de Machine Learning, será avaliada a necessidade de tratamento adicional (como winsorização ou escalonamento robusto) para reduzir impactos indesejados nos modelos.

3.2.1 Atividade A1.3: Testes de Hipóteses ([Link](#))

[]:

A1.3.1: Teste de Normalidade

[]:

A1.3.2: Teste de Independência

[]:

A1.3.3: Teste de Distribuição Como essa variável não é normal, aplica-se somente os testes não paramétricos.

[]:

A1.3.4: Teste das Médias Amostrais

[]:

3.3 Atividade A2 ([Link](#)): Estatística descritiva

3.3.1 Atividade A2.1: Analisar as variáveis Quantitativas

[]:

###Atividade A2.2: Analisar as variáveis Qualitativas

[]:

###Atividade A3: Resultado da Pesquisa de Satisfação

Fazer o resultado da Pesquisa de satisfação do cliente em porcentagem (%): * Geral * Por Sexo

###Atividade A3.1: Discretizar a Variável Qualitativa (OPINIÃO DO CLIENTE)

[]:

###Atividade A3.2: Análise Bootstrap do resultado da Pesquisa de Satisfação

[]:

###Atividade A3.3: Resultado Final da Pesquisa de Satisfação

[]:

4 RELATÓRIO FINAL - PARTE 1

Fazer aqui o **RELATÓRIO FINAL - PARTE 1** com todas as conclusões e resultados que serão apresentados as gestores da **EMPRESA: UVVine**.